

Static and Formal

Analysis that needs no stimulus and proof that needs no test — what each one establishes, where it is a product rather than a craft, and how it joins simulation into one campaign instead of two.

Lecture companion to Chapters 13–16.

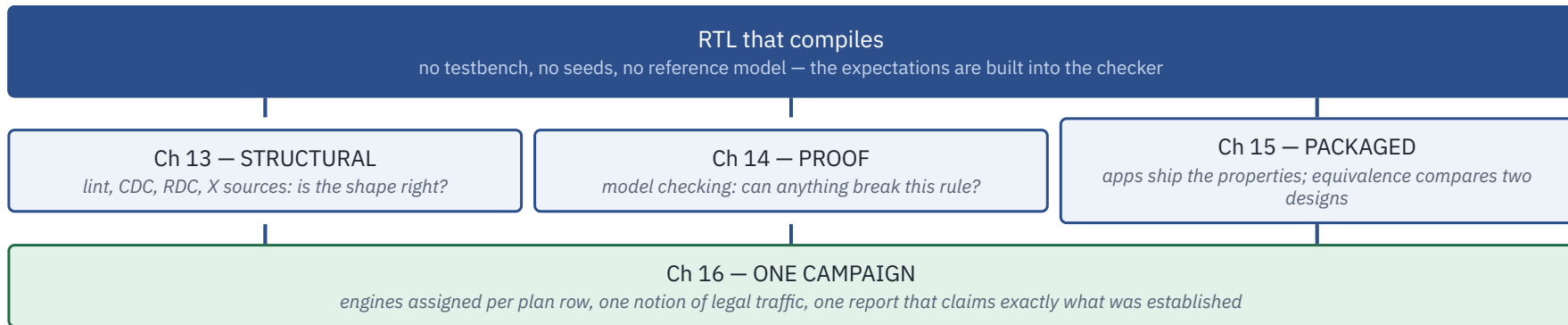
Four chapters, one change of question

A simulator asks *did any run I performed break this rule?* A proof engine asks *can anything break this rule?*

- **What a static gate buys** before a testbench exists — total coverage of a structural question, total silence on functional ones.
- **What “proven” means and what it never means.** A proof is a statement about a model, under assumptions, sometimes to a depth. Drop the three qualifiers and you have a mood, not a claim.
- **Why green is the dangerous colour.** An over-constrained proof passes vacuously and looks exactly like a real one — and it leaves no artifact anywhere except the assumption text.
- **Where formal is industrialised:** apps that ship the properties, and equivalence checking, which has run on every project since long before property checking worked.
- **How to assign a plan row to an engine,** and report the result without inventing a denominator.

SAY *Part III built the instruments that run the design. Part IV is the half that never runs it at all.*

Three engines that need no stimulus — and one chapter that spends them



ENGINE	WHAT IT DECIDES	WHAT IT IS SILENT ABOUT	WHERE THE ENGINEERING ACTUALLY IS
Structural	Whether a recognised structure stands where one is required	Everything the design computes	The setup file: clocks, resets, constants, gray codes
Proof	Whether any legal trace violates a stated rule	Anything outside the model, the assumptions or the bound	Assumptions, abstraction, convergence
Packaged	Design against a machine-readable intent — or against a second design	Whether the intent, or the reference, was right	Curating the intent source

SAY Notice what moves down the table: the effort never disappears, it relocates. That sentence is most of Chapter 15.

13

Static Verification

Exhaustive over structure, silent about function — and a setup file where all the engineering actually lives.

One board in forty hangs on wake-up. The regression is green.

The event unit encodes the wake reason in a **three-bit cause code**, produced on the **32 kHz** always-on clock and consumed at **200 MHz**. Sparse encoding — four legal codes across eight code points — each bit crossing through its own two-flop synchronizer. Three textbook synchronizers, and the linter has nothing to say about any of them.

WHAT THE BOARD DOES

The code changes from `3'b010` (GPIO wake) to `3'b100` (comparator wake), so **two bits change at once** and each destination flop resolves independently. For one system cycle the destination observes `3'b110` — a reserved code the source never produced. The decoder has a branch for each legal cause and *none for the rest*, so it holds its previous decode; the wake sequencer latches the stale value and the handshake never completes.

WHY SIMULATION COULD NOT HAVE SHOWN IT

The source flops change at one instant of simulated time, the destination samples one instant later, and what it samples is exactly what the source drove. **Metastability is a behaviour of a flip-flop with real timing, not of RTL.** Eleven thousand seeds, every wake path the plan enumerated, the scoreboard silent throughout — the design is correct at the level the simulation can see.

The lab is no better: demonstrating this class needs a gate-level model with timing that does not exist yet at RTL sign-off, and the bugs are hard to reproduce once silicon arrives.

WHAT WOULD HAVE FOUND IT, IN SECONDS, BEFORE A TESTBENCH EXISTED

A structural analysis that walks the elaborated design, finds every path leaving one clock domain and entering another, and asks whether a multi-bit crossing is protected against its bits resolving independently. **No stimulus, no reference model, no seed.**

No stimulus, no expected output — and no opinion about function

It analyses the **design description itself**. Chapter 3's two limits do not apply, because it has neither power: it drives nothing, so nothing is unreachable and no coverage question arises; it observes nothing, so nothing it reports is about what the design computes.

WHICH IS WHAT EARNS IT THE FRONT OF THE FLOW

It runs the first day RTL compiles, and it is **deterministic** — no seed, an identical report on unchanged source. That is what makes it usable as a per-commit gate, and as a sign-off row with its own audited waiver file.

AND THE BOUNDARY, STATED EXACTLY

It finds problems deducible from code structure, never problems in the algorithm. **A spell checker finds a misspelled word and will never notice you wrote *with* where you meant *width*.**

PER-COMMIT GATE

Elaboration with the production parameters · lint at must-fix severity only · structural CDC and RDC against a checked-in setup file · nothing else

OUTSIDE IT

Protocol assertions and metastability injection nightly · the formal X-propagation proof as a scoped campaign — never a gate

The setup is the hard part

A structural CDC check is a *setup* followed by a button. Somebody must declare which clocks are asynchronous to which, which inputs are constant in this mode, which signals are pseudo-static, which buses are gray-coded.

DO NOT POINT IT AT THE TIMING CONSTRAINTS

A false path in *timing* means *do not schedule this*. A false path in CDC means **there is no crossing here at all** — a much stronger claim, and usually a wrong one.

“Three clocks” is not “three clock domains”. Had the 50 MHz peripheral clock been the system clock divided by four, the APB boundary would carry no crossing at all. It has its own source, so it does. *Nothing in the RTL distinguishes the two cases.*

It decays through the setup, not the checks: six months on, “the CDC report is clean” means clean under assumptions last examined by someone who has left the project.

From noise to policy — and a report several levels below the bug

```
module wake_decode (
    input logic [2:0] cause,
    output logic [1:0] branch
);
    always_comb begin
        case (cause)           // sparse, no default
            3'd0 : branch = 2'd0; // no wake pending
            3'd1 : branch = 2'd1; // timer
            3'd2 : branch = 2'd2; // GPIO
            3'd4 : branch = 2'd3; // comparator
        endcase
    end
endmodule
```

WHAT THE LINTER SEES — AND WHAT IT DOES NOT

Four of eight values are decoded; for the rest no assignment executes, so `branch` keeps what it held. The report reads *synthesis-intent mismatch, missing `default`*. It does not know that `cause` crosses a clock boundary, or that `3'd6` is the transient the lab is hitting. **Structural, correct, and several levels of abstraction below the bug.**

Four families: synthesis-intent mismatches — the language itself asks tools to flag an `always_comb` that can infer a latch; width and type errors; connectivity; and **constructs whose semantics differ from what the author meant.** `casex` treats `x` and `z` as don't-care in the items *or* the expression, so a runtime unknown on the selector takes a branch nobody chose.

WHY TEAMS DROWN — AND IT IS NOT A TOOL DEFECT

Lint errs on the side of caution, so engineers who chase non-existent problems for a week stop reading the output. **Two multipliers:** replication — four shader clusters report one shared construct once per instance, so *11,000 findings may be forty distinct constructs* — and inherited code, which is somebody else's signal, not noise.

THREE DISCIPLINES, AND ONLY THE FIRST IS ABOUT THE TOOL

Classify by severity before the first run — block / waivable by a named owner / informational. **Filter the output; never disable the check**, and let naming conventions carry the filter so it is mechanical. **Lint as the code is written**, while its author still holds the context.

On an inherited block, do not attempt closure: baseline the report and gate on the *delta*. Retire rule *classes* with a failure mode behind them — never instance counts, which measure elaboration depth.

Metastability cannot be removed — only given time to decay

MEAN TIME BETWEEN FAILURES — READ THE SHAPE, NOT THE NUMBERS

$$\text{MTBF} = e^{(T_s/T_c)} / (T_w \cdot F_c \cdot F_d)$$

Two terms sit in the exponent — the settling window T_s , and the flop's time constant T_c as its divisor. Only T_w , F_c and F_d are linear.

So the first exponential lever is the gap between the first flop settling and the second sampling it — *why two flops work at all*, and why anything eroding it (a slow route, a scan multiplexer, clock skew) hurts out of all proportion to its size.

WHAT IT DOES NOT SAY

Nothing about how often a metastable condition *occurs* — those are frequent. A design does not fail because metastability happened. It fails because **settling did not finish in the window available**, or because there was no synchronizer at all.

THE OPENING SCENARIO, STATED AS A RULE

Fixing metastability and fixing data coherency are **independent problems**. For one bit the coin can land either way and both faces are legal; for several bits you need all the coins to land the same way — so the answer is a gray code, a handshake or a FIFO, *never three correct synchronizers*. Had anyone declared that cause bus gray-coded, the structural check would have accepted one synchronizer per bit and **reported nothing**.

STRUCTURE

Two-flop

one async bit

Gray parallel

one sync per bit

Handshake

multi-bit, any code

Async FIFO

streaming

ITS PRECONDITION

No combinational logic on the path; no pulse too narrow to sample; data stable across more than two destination edges

Exactly one bit changes at a time — so at most one bit goes metastable, resolving to the old code or the new one

Data is not synchronized at all: only request and acknowledge are, and the bus is held stable until the acknowledge returns

Gray pointers both ways, payload through dual-ported memory. Buys bandwidth, and throttling

A clean structural report is half of CDC verification

The structural check asks whether a recognised synchronizer stands where one is required. **It does not analyse functionality**, so a valid structure passes. The other half — that structure's *preconditions* — is an assertion problem.

```
// Wrong: samples the crossing signal on the
// clock that cannot see the problem.
property p_min_pulse_wrong;
  @(posedge dst_clk) $changed(src_sig)
    | => $stable(src_sig)[*2];
endproperty
```

THE FLAW IS IN THE NOTICING

A source pulse narrower than a destination period falls entirely between two destination edges. `$changed` is then never true, **the antecedent never fires, and the property passes having checked nothing** — a clean run reported for exactly the crossing it was written to protect. Only a cover on its antecedent would have exposed it.

```
// Right: sampled fast enough to see a pulse
// the destination could miss.
// N = ceil( 2 * dst period / fast period )
module cdc_pulse_chk #(parameter int N = 2) (
  input logic fast_clk, rst_n, src_sig
);
  property p_min_pulse;
    @(posedge fast_clk) disable iff (!rst_n)
      $changed(src_sig) | => $stable(src_sig)[*N];
  endproperty
  a_min_pulse : assert property (p_min_pulse);
endmodule
```

COMPUTE N, AND THE PROPERTY PRICES THE DESIGN

For the downward crossing — 200 MHz fast clock, 32 kHz destination — **N is 12,500**. The number states in one figure the cost the level-pulse design was hiding, which is why the answer in that direction is a **handshake**.

AND THE SHAPE THAT SURVIVES REVIEW, BECAUSE EACH CROSSING IS INDIVIDUALLY CORRECT

Reconvergence. Two peripheral-domain flags, each properly synchronized, feeding `fifo_empty` && `xfer_done` in the system domain. The synchronizers resolve independently, so the conjunction can assert for a cycle when the source never had both true — or be missed when it did. **The cure is architectural:** qualify one signal with the other, or carry both across a single handshake.

Same clock, different reset: a discipline of its own

An asynchronously reset flop has a third timing requirement beside setup and hold: **recovery time**, between reset release and the next clock edge. It splits in two, and only the second is what the industry means by RDC.

DE-ASSERTION Release lands inside some flop's recovery window sooner or later. Cure: the reset synchronizer — **assert asynchronously, release synchronously**. Purely structural, so static analysis is sufficient.

ASSERTION — RDC A register cleared asynchronously changes at a moment unrelated to any clock edge, and a register *in a different reset domain, and so not being reset*, samples it.

WHY THIS IS NOT A FOOTNOTE TO CDC

The domains in question are reset domains. Both registers can share a clock, so a design with exactly one clock — and not one clock-domain crossing in it — can be full of reset-domain crossings, and *a clean CDC report says nothing about them*.

SCENARIO — THE SENSOR HUB'S ALWAYS-ON ISLAND

Entirely on the 32 kHz clock, with two resets: a power-on reset, and a soft reset firmware pulses to restart the fusion pipeline. **Its CDC report is empty** — one clock, no crossings. In the lab, reconfiguring a sensor occasionally produces a spurious wake interrupt.

Declare the two resets as separate domains and every path into the wake decision is reported. `sample_valid` clears at an instant unrelated to the clock edge; the wake-decision register, in the *other* reset domain and still clocking, samples exactly then. **Neither register is wrong; both resets are correct.**

FIX 1 • ORDER THE ASSERTIONS

If the destination reset is already asserted it samples nothing — which is why the standard carries a *reset assertion sequence* and a *reset group*.

FIX 2 • QUALIFY THE PATH

Block the crossing while the source reset is active. Load-bearing: the qualifier must itself be **synchronous with the destination**.

FIX 3 • SYNCHRONIZE THE RESET

The crossing becomes an ordinary synchronous path. Cheapest of the three here: the island's clock never stops.

RTL simulation lies optimistically and pessimistically at once

A four-valued model driving a two-valued device is lossy in both directions, so a simulation can **pass a test silicon fails** and **fail a test silicon passes**. Which is why the answer is never “make the simulator pessimistic everywhere” — asynchronous reset works *because* of X-optimism.

X-OPTIMISM — THE LANGUAGE'S SEMANTICS, NOT A SIMULATOR DEFECT

An `if` takes the true branch only for a nonzero *known* value; the false branch is taken for `0`, `x` and `z` alike. So **a condition nobody can predict produces a decision nobody chose**, and the run continues as though it were deliberate. `case` and the edge constructs behave the same way.

X-PESSIMISM — THE OPPOSITE ERROR, AND THE VISIBLE ONE

With `a` unknown, both `a & ~a` and `a | ~a` evaluate to `x` where silicon gives a definite 0 and 1. Arithmetic is all-or-nothing: `3'b000 + 3'b01x` is `3'bxxx`.

SCENARIO — THE CRYPTO ENGINE'S UNRESET KEY-VALID FLAG

The negative test that matters most, “*no ciphertext before a key is loaded*”, passes on every seed. The flag powers up unknown, so `if (key_valid)` takes the else branch and the engine refuses to encrypt: **the test passes because of X-optimism, not because of the design**. In silicon the flop powers up definite, and on some corners it is one.

The cure is not a better test but a reset — and in general, attack the sources rather than the propagation. A 2014 case study is the shape: an audio block taken from sixteen channels to seventeen, an index one past the end, and a state machine that X-optimism sent back to idle under a fully passing regression.

ANSWER 1 • GATE LEVEL

Optimism largely goes — but pessimism becomes *more* prevalent, and the unknowns are cleared by depositing known values, iteration after iteration. Arrives late, covers little.

ANSWER 2 • A SIMULATOR'S X-MODE

Merges the 0 and 1 results at each decision, converting optimism into an honest unknown. Costs performance, so in practice a named regression variant rather than a default.

ANSWER 3 • FORMAL X-PROPAGATION

Run blindly it hits the capacity wall: a whole IP on a two-hour budget returned *bounded proofs and no violations* on a design known to contain an X-optimism bug. **Not a wrong answer — an empty one.**

14

Formal Property Verification

Four outcomes, three qualifiers — and one green tick that is not one.

Six weeks of green nights. One number should have stopped the review.

The bound protocol checker from Chapter 11 sits on all nine crossbar ports; its assertions have never fired; the block is a week from sign-off. On the four subordinate-side instances the cover `c_w_overlap` — a second write address accepted at a subordinate while an earlier burst is still unfinished on the write-data channel — reads **zero**. Chapter 11 named that cover as the precondition of exactly one hazard, and in six weeks no test produced it.

SAME FILE, SAME PROPERTIES, DIFFERENT ENGINE

An engineer points a proof engine at one subordinate port instead of a simulator. Overnight it returns **a counterexample eleven cycles long**: two managers have writes in flight to the same subordinate, the subordinate back-pressures the write-data channel, and the crossbar interleaves their beats into one stream. Beats from two bursts are counted as one, and the count no longer matches the length the address at the head of the queue promised.

NOTHING ABOUT THE PROPERTY CHANGED. THE QUESTION DID.

A simulator asks *did any run I performed break this rule?* A proof engine asks *can anything break this rule?* — and on blocks of the right size the second question is answerable in a night.

The same run returned three other things: **two properties proven, one undetermined**, and one proven only because an assumption nobody had reviewed forbade the very traffic that would have broken it. Those four results are the chapter. *The last is the one that will hurt you.*

SAY *Six weeks of green is not evidence of anything until you know which covers were empty while it ran.*

Four outcomes, not two — and three qualifiers on every one of them

Strip the tool away and a synchronous design is a finite transition system. Model checking computes the reachable states by fixed-point analysis and checks the property against that set — an exhaustive search which, **because the state space is finite, is guaranteed to terminate**.

PROVEN

No reachable state and no legal input sequence violates it — subject to the three qualifiers opposite

FALSIFIED

A **counterexample**: the input sequence leading to the failure, with *no irrelevant history attached*

BOUNDED

No violation up to n cycles — a statement about the near future only

UNDETERMINED

The tool gave up: timeout, memory, or an incomplete algorithm

A **cover** behaves symmetrically. Hit: a *witness*. Not hit: *unreachable under the specified assumptions* — or it ran out of memory, *a third answer wearing the second one's clothes*.

THE MODEL

The RTL as elaborated, not silicon. Nothing about the netlist, timing or firmware — and **nothing about what you did not model**: blackbox a submodule and the proof is over the replacement.

THE ASSUMPTIONS

If the real environment violates them the device may fail despite successful verification. The redeeming feature, and it is real: **formal forces them to be written down**.

THE DEPTH

If the result is bounded, the property is guaranteed only within the bound.

An empty cover in simulation means “no test did this”. Unreachable in formal means “nothing can do this”. **Only the second is a statement about the design**.

WHICH IS WHY “THE CROSSBAR HAS BEEN FORMALLY VERIFIED” IS NOT A CLAIM AT ALL

The useful form is longer and duller: *these five properties, on this RTL revision, under these five assumptions, proven unbounded except one which holds to depth 78*. **That sentence is auditable. The short one is a mood.**

One text, unchanged. Two engines, two different questions.

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) +
        ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

SIMULATOR

*did any run I performed
break this rule?*



**evaluated on the bursts the tests
happened to emit**

silent about the ones they did not

PROOF ENGINE

*can anything break this
rule?*



**is there *any legal way* to program
this engine that emits a crossing
burst?**

answered by searching, not by sampling

Now apply the three qualifiers, because they turn a green tick into a sentence you can defend at a review.

MODEL

The midend legalizer and the AXI backend *as elaborated*. It contains the splitting arithmetic where the canonical bug lives — and it does not contain the register frontend, if you blackboxed it.

ASSUMPTIONS

Whatever you said about how descriptors arrive. **Assume a positive stride and you have deleted the negative-stride bug from the model rather than proved it absent.**

DEPTH

If bounded, long enough for a descriptor to be programmed, legalized, split and emitted. A bound that never reaches the first accepted address *proves the property over a stretch of silence.*

Green is the dangerous colour

Assertions and constraints are one artifact with three hats — a generator in Chapter 9, an assertion in Chapter 11, an assumption here. The hats are not interchangeable. **In simulation an `assume` is checked exactly like an assertion. In formal it is never checked: it deletes traces, and the assertions are proved over what survives.**

A MISSING ASSUMPTION ANNOUNCES ITSELF

The model does things the real environment never would, so the engine finds a violation that cannot happen — a **spurious counterexample**. Irritating, costs you an afternoon, *and it arrives in red*.

AN EXTRA ASSUMPTION ANNOUNCES NOTHING

There is nothing to look at. The report is simply greener than before. **That asymmetry is why a sign-off review reads the assume list, not the checkmark list.**

AND FORMAL MAKES IT WORSE IN ONE SPECIFIC WAY

In simulation, deleting a scenario shows up as a **coverage hole**, because coverage is measured on what ran. **In formal there is no stimulus to measure, so the deletion leaves no trace anywhere but in the assumption text.** Over-constraining is usually not obvious and cannot always be found by automatic tools — which is why validating the assumptions is a task in its own right.

The loud grade — the empty model. Two assumptions that contradict each other, or one that contradicts the design's own behaviour. Intuition says every assertion should fail; the opposite happens and *all of them pass vacuously*, because the proof's hypothesis is that all assumptions hold, and that hypothesis is itself false. Cheap prophylactic, applicable by eye: **every assumption should have a design input or a free variable in its support.**

The quiet grade — the satisfiable narrowing. This is the one that reaches sign-off. Assumptions consistent, model far from empty, most properties still proving for the right reasons — and one clause has removed the exact traffic the interesting property was written for.

The property was not so much proved as evacuated

The inline AES-GCM engine on the memory path carries Chapter 11's two properties: **a key may not be loaded while an operation is in flight**, and **ciphertext may not be presented under an unauthenticated key**. Neither proof converges — the key ladder plus the GCM datapath is too much state — so an engineer encodes what the specification's boot sequence says.

```
m_auth_before_load : assume property (
  @(posedge clk) disable iff (!rst_n)
  $rose(key_load) |-> auth_done);
```

Both properties now prove in minutes.

THE DISCIPLINE: DELETE THE ASSUMPTION AND RE-RUN BEFORE BELIEVING EITHER RESULT

This one does not merely shrink the model. It deletes the entire class of traces in which key material enters the engine *unauthenticated* — the class `a_no_ct_unauth` exists to catch.

IN A FUNCTIONAL PROOF

An assumption is a promise about a cooperative neighbour, and the worst case is a *buggy* neighbour.

IN A SECURITY PROOF

The environment is **an adversary who has read your assumption list**. Every assumption is a concession, and each one owes an answer to *why can the attacker not do this?* Here there is none: an attacker able to drive `key_load` is the threat.

WHERE THE OBLIGATION ACTUALLY BELONGED

On the boot ROM's proof, **to be discharged** — not on the engine's, to be enjoyed. Chapter 16 moves it there in three steps, all of them bookkeeping.

SAY The tell is in the timing: a property that would not converge for days proves in seconds shortly after an assumption is added.

One file, two directives, and a record of which side you stood on

```

module xbar_port_checker #(
    parameter bit ASSUME_MODE = 1'b0, // 1 = constrain, 0 = check
    parameter int unsigned IDW = 4, AW = 32
) (
    input logic          clk, rst_n, awvalid, awready,
    input logic [IDW-1:0] awid,
    input logic [AW-1:0] awaddr,
    input logic [7:0]     awlen,
    input logic [2:0]     awsize,
    input logic [1:0]     awburst
);
    property p_aw_stable;
        @(posedge clk) disable iff (!rst_n)
            (awvalid && !awready) | => awvalid &&
                $stable({awid, awaddr, awlen, awsize, awburst});
    endproperty

    if (ASSUME_MODE) begin : g_env
        m_aw_stable : assume property (p_aw_stable);
    end else begin : g_dut
        a_aw_stable : assert property (p_aw_stable);
    end
endmodule

```

Because an assertion in one verification is a constraint in another, **an assumption on a block's inputs should be proven as an assertion on whatever drives them**. That is assume-guarantee: properties of a system deduced from properties of its components, *without ever model-checking the complete model* — the thing that will not converge.

THE PARAMETER IS NOT DECORATION

It records which side of the boundary you were standing on, and makes “*who guarantees this?*” a question with a location rather than an opinion.

ONE THING HAS TO CHANGE FIRST, AND IT IS NOT A PROPERTY

Chapter 11's interface model holds each accepted address in an **unbounded queue** — which a simulator is happy with and a proof engine cannot encode, because termination was guaranteed *by the state space being finite*. Cap it at the port's outstanding-write capacity: a fact about the crossbar, proven on its own and handed back as a **lemma**.

Cannot discharge it? Then check it in simulation — and again in simulation of the larger model the block integrates into. **An assumption that is nowhere checked, in any engine, is a comment.**

Which gives the artifact this section argues for: an assumption register, reviewed like a waiver list. One row per assumption, four columns — the text, who owns the behaviour it describes, how it is discharged, and what breaks if it is wrong.

Two engines behind almost every green line you will read

Bounded model checking — unrolling

For each cycle of an interval of length k it makes a fresh copy of the combinational logic, each copy feeding the next, until the whole finite-time behaviour is *one combinational circuit*. That circuit and the negation of the property go to a satisfiability solver. The one-line intuition: **equivalent to an exhaustive simulation with the same bound on the number of cycles.**

SO A BOUNDED RUN'S SUCCESS IS NOT A PROOF

It says the property cannot be violated *too soon* — and the initial state need not be the reset state. Good tools report “*the assertion has not been violated up to bound 50 clock cycles*” rather than “*the assertion passed*”. **Do not paraphrase that into your own report.**

Push past the **sequential depth** — the longest shortest-path from an initial state to any reachable state — and a bounded result *is* a proof of a safety property. That is usually not available: the solver's cost grows, and computing the sequential depth is itself hard.

Induction — unrolling from an arbitrary state

Stop fixing the starting state: unroll from any state characterised by a predicate P . If every reset state satisfies P , and every interval starting in P both upholds the property and ends in P , the property holds for arbitrarily long time frames — **good forever, from a computation no longer than a bounded one**. The first condition is the one left out of the telling, and it is what keeps the argument from being empty: $P = \text{false}$ is closed under transitions too.

THE PRICE IS A NEW FAILURE MODE, AND IT EATS WEEKS

If P over-approximates the reachable state space, the engine starts intervals in states the design can **never occupy** and produces **spurious counterexamples** from them. The remedy is a *strengthened invariant*: additional assertions that rule those starting states out.

IN PRACTICE THIS IS A READABLE SKILL, NOT A THEORETICAL ONE

When a counterexample arrives, **look at cycle zero first** and ask whether the design could be there at all. A FIFO whose occupancy reads 7 while its two pointers are equal. A one-hot state vector with two bits set. An outstanding count of 3 on a port that holds at most 2. Each observation becomes a helper assertion — proven on its own, then supplied to the engine as an invariant.

Same block, same run, same number — two different amounts of evidence

The rule of thumb — a bound a little larger than the pipeline depth — misleads, because most interesting properties are not pipeline-shaped. **Do the count.** One run on the DMA backend, bounded at **40 cycles**, two of Chapter 11's properties on it:

A_NO_OVERFLOW — 40 IS A GENEROUS BOUND

It guards a datapath FIFO of **16 entries**. Filling it takes at least sixteen pushes with no intervening pop, so the interesting state is reachable well inside forty cycles — and the paired `c_full` cover confirms it with a witness.

A_W_BURST_LEN — 40 IS A STRETCH OF SILENCE

It reconciles a burst's beat count against the length its address promised, and can only do so at `WLAST`. Maximum burst is **256 beats**, so that `WLAST` cannot arrive until the 256th data beat has been handshaken. Inside forty cycles no burst longer than about forty beats completes at all: **proven for short bursts, completely silent about long ones.**

AND ONE CYCLE EITHER SIDE OF A BOUND CAN BE THE WHOLE RESULT

This chapter's opening counterexample is **eleven cycles long**. A run stopped at ten would have reported *not violated up to 10 cycles* — **green, filed, and wrong.**

HENCE A DISCIPLINE THAT COSTS ONE EXTRA DIRECTIVE PER PROPERTY

Before trusting a bounded pass, run the property's antecedent — or the interesting state it guards — as a `cover` under the same bound. A cover landing below the bound validates it; a cover with no witness inside means the proof is a statement about a stretch of silence.

Not an advanced topic. The technique — with a knife edge.

For a proof to complete, the model must be small enough to finish in the CPU and memory available. Every reduction changes the set of behaviours the model can exhibit, and **the direction of that change decides which results you may still believe.**

OVER-APPROXIMATION — ADDS BEHAVIOURS

The abstract model allows *more* traces, so a proof on it holds on the original: over-approximation is **sound**, and a success is always accurate. A failure may be *spurious*.

UNDER-APPROXIMATION — REMOVES BEHAVIOURS

A failure is always real, so it is **safe**. But a pass means nothing — exactly why such tools say “*not violated up to bound 50*”.

The practical hazard is human, not logical. Someone who has waded through twenty impossible traces starts skimming. *The twenty-first is real* — the capacity a reduction buys is paid for in the analysis of spurious failures.

MANUAL LEVER

Free a signal

a cutpoint; blackboxing is the same lever on every output at once

Set an *input* constant

Set an *internal* signal

Uninterpret

memory, wide arithmetic

Tools prune the logic outside a property's cone of influence on their own; manual reduction is for signals that affect it indirectly, which is why it takes detailed knowledge of the design.

DIRECTION

Over-approximation: **sound, not safe.**

The cut makes the result *stronger*, not weaker — proven for every value includes the correct ones

Under-approximation: **safe, not sound**

Neither safe nor sound. Every use owes a sentence in the record

A box whose only property is that equal inputs give equal outputs

Two caches, one address — and the list you write at the moment of the cut

The coherent hub is the standard teaching case because its state space is a product of things you cannot bound: the MESI state of every line, in every cache, for every address. Chapter 11 left one invariant with an explicit promise — **at most one cache may hold a line Modified**:

```
a_single_writer : assert property (
  @(posedge clk) disable iff (!rst_n)
  $onehot0(modified_vec));
```

WHAT SURVIVES A CUT TO TWO CACHES AND ONE ADDRESS

The directory's transition rules for a line, **every race between two requesters** for it, the ordering of invalidations against data responses, and — since livelock lives on a single contended line — **starvation of one requester by the other**. None of it is reachable by simulation at any run length you can afford.

WHAT DOES NOT SURVIVE — WRITE IT DOWN NOW, NOBODY RECONSTRUCTS IT IN THREE MONTHS

A second address: the classic coherence deadlock — two agents acquiring two lines in opposite orders — is invisible by construction. **Capacity**: with one address there are no conflict evictions. **A third sharer**, including the sharer-vector encoding and its overflow.

AND A WARNING ABOUT THE CUT ITSELF

Cutting many caches down to two is **not a sound abstraction** in the sense opposite: freeing and setting have a defined direction of error, and *deleting requesters has none*. It rests on a **symmetry argument** — that the hub treats its requesters interchangeably — which is a claim about the RTL, not a theorem. If the directory arbitrates fixed-priority by requester index it is simply false.

THE GENERAL ANSWER TO “HOW DO I AVOID THROWING AWAY THE BUG”

You cannot, entirely — an abstraction that kept everything would not converge. What you can do is **name the bug classes it deletes, at the time you delete them, and give each one somewhere else to be caught**: the two-line deadlock to a separate proof, the eviction path to system-level simulation with a cache-thrashing profile, the sharer-vector overflow to a directed test. Each becomes a plan row reading *not covered by proof P-COH-2 — covered by...* **A proof whose deletions are enumerated is evidence; a proof whose deletions are implicit is a slogan.**

A cheapest-first order — not the order the menu is written in

Sooner or later a run comes back neither green nor red. Getting a conclusive answer means refining the verification model, and **the order you work through the options is what separates an afternoon from a fortnight.**

1 Ask whether the property is even true. Run bounded, run deep. An undetermined property is sometimes a *false* one whose counterexample sits further out than the last bound you tried — one overnight job instead of a week of strengthening.

2 Ask whether it was written for this engine. In simulation an inefficient assertion costs runtime; in formal it can be fatal. Where one text must serve both engines, *the formal-efficient form should win.*

3 Split it. A property asserting five things at once is five proofs sharing one worst case. Separated, the report names the one that did not close instead of hiding four successes behind it.

4 Prove lemmas and feed them back. A queue's occupancy matches its pointers; a state vector is one-hot; an outstanding counter never exceeds capacity. *The only kind of assumption that costs nothing, because you have discharged it.*

5 Case-split, once you have shown those configuration values partition the space — N results, and the plan row must say so.

6 Abstract — last of the model-changing moves, because the deletion list comes with it.

7 Accept a bounded result, with the depth recorded and its antecedent covered under the same bound.

8 Stop. Hand the plan row to simulation and close it there.

Four of them — rewrite, lemmas, case-split, abstract — change the problem instead of answering it, and return you to the top with a smaller one. Step 8 is a legitimate outcome, not a defeat: a team that cannot reach it will spend a whole schedule on one property.

Coverage when there is no stimulus — and the one measure that is not structural

MODEL

THE QUESTION IT ANSWERS

Cone of influence

Did anybody write a check about this logic at all? Cheap, purely structural — but a fully proven property does not mean its whole cone was verified

Input stimuli

Ignores the assertions; measures what the *constraints* let the engine reach — **the silent over-constraint hazard as a number**

Bounded proof

How much of a cone the bound reaches. Two days of engine time between bound 10 and 11 adding nothing new: **stop buying depth, start abstracting**

Proof core

The logic actually used can be far smaller than the cone, and **a bug outside the core but inside it goes uncaught by a fully proven assertion**

All four are structural — they say which logic was involved, not whether a check would have *noticed a change* to it. The empirical complement is **mutation**: break the design deliberately, and some assertion should fail.

THE MOST CONVINCING PUBLISHED RUN — A DAC CHALLENGE, 2015

Attendees inserted **73 functional bugs** into a crossbar that already had a formal testbench. **Every one triggered a failure** of one or more checkers.

READ THE FOOTNOTE AS CAREFULLY AS THE HEADLINE

Two further changes failed nothing: one had *no functional impact at all* (a loop bound rewritten to save area), the other was inserted *specifically to evade the methodology* — a non-bug, because a determined adversary always can. **Mutation grades a property set against the bugs designers make, not against an attacker.**

The review reads the assume list, not the checkmarks

PROPERTY (BINDING)	RESULT	DEPTH	ASSM
a_aw_stable 4 sub-side	proven	unbounded	A2
a_b_id_outstanding	proven	unbounded	A2, A4
a_w_burst_len	falsified → fixed → proven CEX 11 cycles	unbounded	A2, A4
a_grant_within_64 per manager port	bounded pass	78	A1, A2, A3
a_b_order	proven under A5; <i>undetermined</i> <i>without it</i>	—	A2, A5

A3 • DISCHARGE: SIMULATION ONLY

No manager stalls its own responses indefinitely. A known concession, accepted with a written argument — *the discipline working*.

A4 • DISCHARGE: PROVEN ON THE CROSSBAR ITSELF

Handed back as a **lemma** bounding the checker's address queue. Two managers with one write each still overlap, *so the opening's bug stays in the model*.

A5 • DISCHARGE: NONE

Added mid-debug to make a_b_order converge, never reviewed — and **doing the proving**: with one response per ID in flight there is no pair left to order. *Withdrawn at sign-off*.

An assume column is derived per row. One filled by copying is the same defect as an undeserved checkmark, a table to its left.

ROW FOUR IS THE ONLY BOUNDED RESULT, SO IT OWES A DERIVATION A REVIEWER CAN RECOMPUTE

Round-robin serves each of the other four managers at most once ahead of this one, and each holds the target's address port for **at most sixteen cycles** (by A1), so the grant cannot fall later than $4 \times 16 = 64$. Its antecedent — five requests pending at one subordinate — is first witnessed by a cover at **cycle 12**, so the shortest counterexample it admits is **76** cycles; two cycles of margin put the run at **78**.

15

Formal Apps and Equivalence

Formal packaged as a product rather than practised as a craft — and the checking flows that have run quietly on every project for decades.

The interrupt arrives on input 5. The header says input 6.

Two thousand lines of instantiation and wiring at the top level, containing one mistake. **Both blocks are correct**, so every block-level environment passes. The system regression passes too, because no test both provokes an accelerator error *and* enables that particular interrupt: the error path is exercised by the accelerator's own tests, which read the status register, and the interrupt map by the event unit's tests, which inject events directly.

IT SURFACES IN THE LAB, AS A HANG

Nothing here is subtle. The intent is **a table with a few hundred rows** — this source, that destination, under this condition — and the implementation is structural wiring. Comparing the two is mechanical, and answering it by simulation is absurd: one directed test per row, each proving a single wire, none exhaustive over the conditions.

MEANWHILE THE TEAM'S ONE FORMAL SPECIALIST IS UNAVAILABLE

She is three weeks into the crossbar's ordering proof, choosing abstractions and defending assumptions. **She is also not needed.** This question requires nobody to write a property. It requires a tool that already knows which property to write, and *a table to write it from*.

SAY Every integration bug of this shape survives block-level verification by construction, because every block is correct.

The tool brings the properties. You bring the design and a configuration.

Chapter 14's flow is mostly specification work — read the spec, decide what is worth proving, write properties, state and defend assumptions, fight convergence. **Only one of those steps is the engine's.** An app inverts the economics by *shipping the properties*: an X-propagation app reads the RTL and implements the assertions itself. **Nobody writes a property or states an assumption.**

APP	WHAT IT PROVES	WHERE ITS INTENT COMES FROM
Connectivity	Every declared connection exists, and no undeclared one does	An integration table
Register access	Reset values, access policies, decode	A register description
Unreachability	A coverage target admits no legal trace	The coverage database
Auto-generated	Interface and structural invariants extracted from the RTL	<i>The RTL itself</i>
X propagation	No unknown reaches a control point	Reset and initialisation structure (Ch 13)
Clock-domain crossing	Synchroniser structure and protocol correctness	Clock and reset declarations (Ch 13)

AN APP RELOCATES THE SPECIFICATION — IT DOES NOT REMOVE IT

Properties come free only where the intent is already machine-readable. **Every hour saved on property writing reappears as intent curation.**

AND CONVERGENCE DOES NOT TRANSFER

Whole IP, two-hour limit: no violations, and only *bounded* proofs. **An app makes property writing free. It does not make the design smaller.**

Adoption: automatic formal apps grew 62 percent from 2012 to 2014 against incremental growth for property checking, and the gap persisted — IC/ASIC 2016-2024, 5.8 percent compound annually against 8.7 for automatic apps; FPGA, 7.5 percent for formal together.

Proving the wiring — the highest-value, lowest-effort app on most projects

```
// Intent row: src = nn_err_irq, dst = evt_in[6],
//           enable = cfg_irq_en[6], latency = 1 cycle.
property p_conn_nn_err;
  @(posedge clk) disable iff (!rst_n)
    cfg_irq_en[6] | => (evt_in[6] == $past(nn_err_irq));
endproperty
a_conn_nn_err : assert property (p_conn_nn_err);
```

The antecedent is the enabling condition sampled now; the consequent fires one tick later and compares the destination against the source *as it was when the enable was true*. It says nothing about cycles where the enable is low — an unconditional claim would be wrong. **Two hundred rows of this, generated and discharged in one run, is the entire flow.**

ONE OF THE TWO HUNDRED IS THE ROW ABOVE

It does not prove: the wiring reaches input 5. **No test had to provoke an accelerator error while that interrupt was enabled** — the proof does not care whether one ever would.

THE DUAL CLAIM, AND THE PART TEAMS FORGET

A table listing what must be connected says nothing about what must **not** be. On the crypto engine: no path from any key-ladder output to a bus-readable address, the debug data register, or a scan-observable point. Cheaper still — a structural question a tool answers without a proof engine getting involved.

BUT BE EXACT ABOUT WHAT A NEGATIVE ROW PROVES

A no-connection result is *structural*: no path exists. It does not say the key never influences an observable value along a path that *does* exist — that is an information-flow question. **Confusing the two is how a team believes it has proved non-leakage when it has proved non-adjacency.**

Scaling. Make the port indices *symbolic* — an arbitrary manager and subordinate, constrained only to be stable — and one property covers every value they could take. Not always faster: **162 seconds symbolic against 65** for the exhaustive per-path proofs together.

The map against its description — and the gap it does not close

One register description generates every view a project already has and already lets drift: RTL decode and field logic, verification model, firmware header, documentation. **The app closes the remaining edge** — it proves the RTL implements the description, per register and per field.

FIELD `OE`, THE UART RX
OVERRUN FLAG

VALUE

width · lsb

1 · 3

access policy

"W1C" — write one to clear

volatile · reset · has_reset

1 · 1'b0 · 1

A conventional register test walks the map front-door, pattern by pattern — which is why a large map's register test is measured in hours. The app proves the same statements **over all values and orderings**, at RTL rather than through the bus.

Scenario — the GPU's four shader clusters carry identical register banks; cluster 3 is hand-edited during a late change and one counter's reset value is left at the previous revision's. The app enumerates from the description, so it checks cluster 3 whether or not anyone thought about cluster 3.

TWO OF THOSE ARGUMENTS ARE EXACTLY WHAT THE APP IS FOR

"W1C" is a statement about **every possible write value and both possible current values**, of which a random register test exercises a handful. And `volatile` records that hardware can change the field without software's involvement — so the check must read *a software write of 1 clears the field unless hardware sets it in the same cycle*, not *the field is 0 after the write*. **An app that ignores the flag generates a check that fails on correct hardware**, and the first thing a team does with a failing generated check is switch it off.

NOW THE HONEST LIMIT

The app proves *RTL against description*, and the register model and the RTL check come from the same file: **a common-mode error, encoded once and read twice**. Nothing here reviews whether bit 3 was the right bit. That review is a human reading the description against the architecture document — and the app's value is that it removes everything else.

A proof instead of a waiver — and what the verdict inherits

A coverage exclusion's reason is almost always an engineer's claim that a state is unreachable. That is a reachability question about the design, and reachability is what a model checker computes: **what was an opinion in a waiver becomes a result in a database.**

CATCH ONE — THE WORD *LEGAL*

Reachability is computed relative to the constraints, and a wrong constraint makes reachable behaviour look impossible. One case is worth memorising because it is so ordinary: an input constraint hiding in a tool setup script, `enable_cnt == 0`, made a counter's increment line uncoverable. **It was reported unreachable, and it was** — under an assumption nobody had reviewed.

CATCH TWO — IT WORKS ON THE *DESIGN*

RTL lines, branches, FSM states, cover properties bound into design scopes. **A functional coverage model computed by the testbench is invisible to it.**

BE PRECISE ABOUT WHAT THAT DOES AND DOES NOT OVERTURN

The exclusion is not refuted. What is refuted is its *type*. It looked like a statement about the design and it is a statement about the stimulus — true while the generator stays as it is, false the morning someone widens it. So the disposition stays a waiver, and *this is what its expiry field is for*: the expiry is an edit to the constraint file, which nobody counts as verification evidence until the morning it is.

```
// Can the midend ever emit a full-length burst out of a
// transaction it had to split at a 4 KB frontier?
c_split_max_len : cover property (
  @(posedge clk) disable iff (!rst_n)
  midend_txn_done && midend.did_split
  && (midend.burst_len == 9'd256));
```

The canonical DMA cross `x_worst` has **72** combinations. **Six** — the whole `straddle × max` column, 3 stride signs $× 2$ other-channel states — are excluded by the generator's `c_solver_budget`, a budget the team *chose*. That leaves **66** cells to close.

THE ENGINE ANSWERS IN SECONDS — AND THE COVER IS REACHABLE

Nothing in the design bounds a descriptor row at one maximum burst. **The bound that empties the column lives in the constrained-random generator, and a design-level cover does not inherit it.**

A draft, not a specification

Point this app at the RTL and it extracts implementation-detail properties — interface handshakes, one-hot encodings, occupancy bounds, the structural invariants a designer would state if anyone asked. No table, no description, no configuration beyond a module name. **Judged by the intent column it is the cheapest row in the catalogue: the intent source already exists, and it is the design.**

WHICH IS EXACTLY WHAT IS WRONG WITH IT

Every other app reads intent from something that is *not* the design — a wiring table, a register description, a coverage model — and **the gap between the two is where the app finds bugs.** This one closes the gap by construction.

The limitation is structural rather than a matter of tool maturity: mining generates and evaluates its properties on the same RTL *with no golden reference to appeal to*, so flaws in the RTL yield flawed properties and the method has nothing with which to notice.

FOLLOW THAT THROUGH

A mined property **cannot fail on the RTL it was mined from.** It proves — and it would have proved on a design with the opposite behaviour, because the behaviour is where it came from. *Its bug-finding power against today's RTL is zero, and no amount of proof depth changes that.*

WHAT IT IS WORTH IS WHAT A REGRESSION IS WORTH

It converts the design's present behaviour into an obligation, so the next edit that changes that behaviour has to **argue with something** instead of sliding through.

The review question for each of three hundred generated properties is not *is it true* — it is, by construction — but *is it intended*, and answering that is specification work, property by property, done by the specialist the app was supposed to make unnecessary. *An app that emits verdicts removes that work. An app that emits drafts relocates it.* **Budget the review, or do not run the app.**

SCENARIO — THE INHERITED UART, FOUR YEARS OLD, SPECIFICATION LOST

Its RX overrun flag is never cleared when software reads the status register in the same cycle a stop-bit error arrives. The RTL exhibits it, so a mined property states it **as an obligation** — and the day someone fixes the bug, the property fires as a regression. *A generated set encodes the design's defects with exactly the fidelity it encodes its features.*

The flow's quiet workhorse: a design against another design

Every technique so far compares a design against a *specification*. This compares a design against another design — weaker-sounding, worth more, because the flow's most frequent transformations are ones nobody verifies any other way. Chapter 2's reconvergence model: synthesis turns RTL into a netlist, equivalence checking is **an independent second path from the same origin**, and the two reconverge at a proof that the transformation preserved functionality. *No stimulus, no coverage, no oracle problem.*

LEC — AND ITS ASSUMPTION IS THE WHOLE STORY

It does not reason about sequences. It matches *state points* — the registers of both models — one-to-one, then proves the combinational logic computing each is Boolean-equivalent. Everything follows: fast, exhaustive because it never enumerates a trace, and it **degenerates into a mapping problem** when the state points do not match.

It keeps the synthesis tool honest. The canonical illustration: a design synthesised incorrectly from a synthetic arithmetic operator whose documentation said correctness was not guaranteed above **48 bits** — documentation the synthesis tool could not read. Equivalence checking found it quickly; gate-level simulation with great difficulty, if at all.

SEC — DROPS THAT ASSUMPTION AND PAYS FOR IT

Equivalence now means *every input sequence produces the same output sequence*, decided on a product machine running both designs in lock-step. **That is a model-checking problem, with model checking's costs** — so it is aimed at bounded, well-chosen transformations rather than sprayed across a design.

Clock gating. Gates inserted on the accelerator's 16-lane MAC datapath: if the condition is wrong by one cycle the datapath holds a stale value that surfaces only on a weight-stream boundary. **Retiming.** The radar front-end's FFT pipeline misses timing, a stage boundary moves — same arithmetic, different register mapping, so LEC will not map the state points at all.

AND ONE ROW HOLDS ACROSS LEC, SEC AND THE DATAPATH FLOW ALIKE: WHAT EACH IS BLIND TO

Whether the *reference itself* is right. **Equivalence is a relative argument: it transfers your confidence in one model onto another exactly and without loss, and it transfers your errors with the same fidelity.** Nothing here says the RTL is correct. It says the netlist is as correct as the RTL — a smaller claim, and the one the flow actually needs.

Where simulation's coverage argument does not degrade — it collapses

THE CLEAN CASE: A CODEC'S INVERSE TRANSFORM

An 8×8 residual block is **64 coefficients**; at 16 bits each, the input space of a *single invocation* is 2^{1024} values. No coverage model samples that meaningfully, and no argument from “we ran a billion random blocks” survives contact with the arithmetic.

Worse, the failure mode is not a region of the space but a scatter — a rounding term applied at the wrong stage, a saturation clipping one bit late, an intermediate width one bit short. Each is wrong for a sparse, structured subset of operands that random stimulus meets with probability near zero. And the standard specifies the transform as *exact integer arithmetic*, so a one-bit mismatch in one coefficient is a conformance failure, not a quality issue.

SCENARIO — THE RADAR FRONT-END'S CFAR DETECTOR, AND THE COMPARISON NOBODY ASKS FOR

Datapath verification proves the RTL bit-exact against the algorithm team's fixed-point C model, on every operand. **Ship it — and then run the other comparison:** the fixed-point model against the floating-point one, over the operating envelope, with a declared error bound. Bit-exactness against the fixed-point reference is a complete answer to *did we build the model correctly* and **no answer at all to is the model adequate**. A detector that saturates on a strong near-target loses a weak far one, and every stage of the equivalence argument passes while it happens.

WHAT MAKES IT TRACTABLE: THE SPECIFICATION IS NOT PROSE

It is a **mathematical relation between operands and result**, and it can be written as one. The pattern established after the floating-point division episode: state a multiplier's top-level specification as a formalisation of the arithmetic standard, with significand and exponent read as *integer values rather than bit vectors*.

And **decompose**: over **120 properties** covered all arithmetic operations of one industrial floating-point unit, each a property about a subunit, with the irrelevant parts extracted away per property. Two lessons carry: *reason at word level, not bit level*, and *the monolithic claim does not converge*.

16

Hybrid Flows

One design, one plan, one sign-off — and two engines that must stop being run as two projects.

One dashboard number. Ask what its denominator is.

The flagship's host cluster has been in verification for a quarter, and two engineers have been working on it without reading each other's reports.

THE FORMAL REPORT SAYS

Nine properties on the directory's transition rules, proven unbounded on a model cut down to **two caches and one address**. An assumption register of **five rows, four of them discharged** on a neighbouring block. Among the nine is the invariant that at most one cache may hold a line Modified.

THE SIMULATION REPORT SAYS

Four hundred runs of random traffic across four application cores. A functional cross of MESI state against requester against concurrent-snoop standing at **71%**. A scoreboard silent for three weeks — and the note that the remaining holes are all in the *eviction column*.

THE LEAD'S DASHBOARD MERGES BOTH DATABASES AND PRINTS ONE NUMBER

That number is not wrong about anything, and **it is not a claim about anything either**. Ask the sharper question — *does the proof of the single-writer invariant fill the cross bins that ask the same question?* — and you get two confident, opposite answers from two competent engineers. **Neither of them is doing anything wrong. The flow is.**

SAY There is one design, one plan and one sign-off. The two engines have been run as two projects that meet for the first time at a dashboard.

Assign engines to plan rows, on properties of the problem

PROPERTY OF THE ROW

HOW TO READ IT

How the criterion quantifies

For every burst, grant or line — a claim over all traces, which a sampling engine can only corroborate. *For this traffic* — a claim about a transformation, whose oracle is another executable

Corner or volume

A *corner* is a conjunction of rare conditions: a product of probabilities in simulation, a search that ignores rarity in formal. A *volume* — throughput, occupancy, fairness — is statistical, and there is nothing there to prove

What a witness costs

A counterexample is near-minimal with no history attached; a simulation failure arrives with everything that preceded it. Criteria over internal state are cheap in *both* engines — which is why Chapter 11's invariants have a second career as formal lemmas

SCENARIO — TWO GPU ROWS ON ONE INTERFACE

Row one: *no shader cluster may observe a write reordered past a barrier it issued* → **formal**, on the one-symbolic-address memory model. A corner: one specific interleaving of barrier, write and cross-cluster read, whose probability under random stimulus is a product of three small numbers.

Row two: *at 90% memory-bus utilisation the 99th-percentile shader stall stays under N cycles* → **simulation**, with a real-frame traffic profile and a distribution for a closure metric. Ask a proof engine “*can this stall exceed N ?*” and it returns a worst case that exists, occurs once in a billion frames, and was never what the row was about.

AND THE ASYMMETRY THAT BELONGS IN THE PLAN, NOT IN WEEK NINE

When the answer is *no violation*, **simulation's cost is predictable and formal's is not**. A few minutes of simulation gives a good estimate of clock cycles per second for the rest of the run; if an assertion is slow to prove, it is almost impossible to predict how much longer it will take.

So a lead may put a simulation row on the critical path and estimate it. A formal row there is an **unbounded commitment** unless the plan says in advance what happens when it does not converge — which is why the convergence ladder ends with *stop*, and *hand the row to simulation* as a **planned outcome**.

One interface rule lives three lives — and the dangerous drift is silent

GENERATOR CONSTRAINT

a predicate over a randomized object — produces legal traffic on the block's input

ASSERTION

a predicate over a signal history — checks legality on the neighbour's output

ASSUMPTION

the same text, deleting traces from a formal model

These are **not one statement in two syntaxes**. So no tool diffs them, and no compiler complains when they diverge. Teams discover the divergence at sign-off, or never.

DRIFT ONE — THE GENERATOR IS NARROWER THAN THE ASSUMPTION

The formal environment permits traffic the tests never send. **This is sound:** the proof covers a superset of what simulation exercised, so nothing is claimed that is not established. And it shows up honestly, *as a coverage hole on the simulation side*.

DRIFT TWO — THE ASSUMPTION IS STRONGER THAN THE GENERATOR'S LEGAL SET

The proof holds over a *subset* of the traffic the tests actually send — so simulation is exercising behaviour the proof never covered, **while both reports are green**. Nothing on either side is empty. There is no hole, no failure, and **no artifact anywhere that records the gap**.

This is Chapter 14's quiet over-constraint arriving through a second door: not an assumption written too strong for the design's environment, but one written accurately for an *older* environment that the stimulus has since outgrown.

Make the assumption the source of truth — it is the one whose failure is silent

```

module wake_handshake_checker #(parameter bit ASSUME_MODE = 1'b0) (
    input logic clk_slow, rst_n, wake_req, wake_ack
);
    // The requester may not withdraw an unacknowledged request.
    property p_req_holds_until_ack;
        @(posedge clk_slow) disable iff (!rst_n)
            (wake_req && !wake_ack) |>= wake_req;
    endproperty
    c_req_waited : cover property (@(posedge clk_slow)
        disable iff (!rst_n) wake_req && !wake_ack);
    if (ASSUME_MODE) begin : g_env
        m_req_holds_until_ack : assume property (p_req_holds_until_ack);
    end else begin : g_chk
        a_req_holds_until_ack : assert property (p_req_holds_until_ack);
    end
endmodule

```

Read the property before believing anything about it. It says only that an *unacknowledged* request survives into the next cycle — nothing about life after `wake_ack` rises, and **it never constrains `wake_ack`**.

DETECTOR ONE — COMPILE THE ASSUMPTION SET INTO SIMULATION AS ASSERTIONS

When the DSP's wake controller is proved, `wake_req` is an input and the instance is built with `ASSUME_MODE = 1`. When the island is proved, the same text is *asserted*. And bound into the DSP's own block-level simulation, where `wake_req` comes from the testbench, **that instance is checking your generator, not the design**. If a sequence ever drops an unacknowledged request it fires, naming the assumption that would have been *silently violated* in formal — on a nightly run rather than at sign-off.

DETECTOR TWO — COVER THE ASSUMPTION'S BOUNDARY

Detector one catches the generator going *outside* the assumption. It says nothing about the commoner case: staying so far inside that the assumption **was never tested**. If `c_req_waited` reads zero, the assertion has never evaluated non-vacuously. **Every assumption ships with a cover on the condition it restricts.**

Together the two turn Chapter 14's assumption register from a reviewed document into a monitored one — each row gaining a column for its assertion's last pass and its cover's last hit.

What a unified report may claim, and what it may never

A simulation bin holds a count: *this situation occurred, n times, in the runs I performed*. A proof holds a claim: *no legal trace violates this rule*. They are not two measurements of one quantity — one is a statement about your testbench and the other about your design. **Average them and the percentage has a denominator made of two incompatible units.**

MERGES IN ONE UNIT	TRAVELS AS A TAGGED ITEM, WITH ITS QUALIFIERS	MUST NEVER BE SUMMED
Code coverage — lines, branches, conditions: formal reachability and simulation report hits against the same structural denominator	An assertion's result plus its radius — a bounded proof's depth or a counterexample's length — with the assumptions, abstractions and cutpoints	Functional bins against proofs. A hit count and a proof answer different questions
Assertion coverage: the same property's pass and vacuity counts across simulation runs	An unreachability verdict on a coverage target, <i>with the constraint set that produced it</i>	Two percentages whose metrics partition the space differently
Plan-row status: each row closed by <i>its</i> declared metric, whichever engine produced it	A merged view — provided it is recorded as a <i>derived</i> item and labelled as one	A proof's “100%” into a functional bin the tests never filled

THE STANDARD'S OWN CAUTION ON HETEROGENEOUS MERGES

Such items are *more often semantically different than equivalent*, so they should not be merged automatically; a tool that does must record the result as a **new item carrying a semantic tag saying it is merged data**. The merging algorithm itself is left unspecified. A *licence with a condition attached*.

AND A PERCENTAGE IS A PROPERTY OF A MODEL, NOT OF A DESIGN

An eight-bit variable that took the values 17 and 96 scores **2/256** under a metric with one bin per value and **2/8** under eight-bin auto-binning. *Collection loses information that cannot be recovered afterwards* — and it compounds when the denominators come from two engines that never agreed on a partition.

Unify at the verification plan, not at the coverage database

The honest answer is not a better merge algorithm. Chapter 5's **one-row-one-metric** rule was written for exactly this: a row discharged by a proof closes on the proof — with its depth, its assumption list and its own formal coverage evidence — while a row discharged by a covergroup closes on the covergroup. *Nothing has to be averaged, because nothing was ever a fraction of the same whole.*

WHAT THE DASHBOARD REPORTS

Rows closed out of rows planned, per engine and in total — a number with a denominator anyone at the review can name.

That is also the honest form of “*proven, so no coverage needed*”: the row needs no functional **bin**, because the proof discharges what the bin was sampling towards — and it *still* owes the formal coverage models, which is why a proof never closes a row on its own.

JOB ONE — STRUCTURAL COVERAGE, IN ONE UNIT

Formal reachability produces line, branch and condition coverage in precisely the units simulation uses, so those genuinely add. On a published multicast crossbar of eight clients and eight targets, the *completed formal testbench alone* covered **all 288 lines and all 88 conditions, deepest cover point at depth 5** — inside that project's derived required proof depth, which the authors take as evidence the depth is not optimistic.

JOB TWO — THE TRIPWIRE TEAMS LEAVE SWITCHED OFF

An item can be flagged **formally unreachable**, and the classification separates a don't-care from a *predicted-unreachable* point precisely so **a warning is raised if it is later covered**. Chapter 15 turned a waiver's argument into a proof; this makes the proof a live tripwire.

AND THE FAILURE WORTH NAMING, BECAUSE IT LOOKS LIKE PROGRESS

Writing a proof's verdict into a functional bin destroys two instruments at once. The bin's job is to record *what the stimulus did*: overwrite it and the fresh-seed yield no longer discriminates a converged suite from a saturated one, while the coverage differential can no longer fail a commit that quietly made the suite ask less. **You have not gained coverage. You have blinded two metrics to buy a number.**

A decision procedure a lead can apply in a plan review — per row, not per block

- 1 Does the criterion quantify over **all traces**, or over a **traffic profile**? Profile criteria stay simulation rows however elegant the property would look
- 2 Is the interesting behaviour a **corner** or a **volume**? A conjunction of rare conditions goes to formal even when the block is large, because the abstraction that makes it converge usually preserves the conjunction
- 3 Can the block's environment **even construct the witness**? If not, the row is not a stimulus-tuning problem
- 4 What does the row's model have to **contain**? Price the abstraction bill and the deletion list before committing
- 5 **Who owns sign-off for this row?** Sometimes the answer is nobody on your project
- 6 What happens **if it does not converge**? Answer this in the plan review, not in week nine

QUESTION FIVE — THE USB CONTROLLER'S LINK-TRAINING ROWS

The U0 through U3 transitions are as good a formal target as exists, and a proof of them is cheap, fast and finds real bugs. **Run it, and do not let it close the rows** — their closure metric is the specification-mandated compliance suite's item list. *Cheaper but not sufficient for sign-off* is not a criticism of formal: the proof is **stronger** evidence than the suite for the rules it covers. It is a statement about who holds the authority to declare the row done.

AND THE OUTCOME THAT LOOKS LIKE WASTE AND IS NOT: BOTH

One deployment began its formal effort *after* a major dynamic-verification milestone had completed, on designs whose simulation environments were already stable — precisely where a second method should find nothing — and reported **82 interesting bugs over twelve weeks**. The taxonomy is the useful part: *easy*, a state machine attempting two arcs at once; *medium*, counter overflows and arbiter starvation; *hard*, a deep internal counter value coincident with a specific input sequence. Even the easiest would have been difficult for the running environment to find.

Artifacts crossing between two engineers — and what must travel with each

Formal → simulation

A COUNTEREXAMPLE BECOMES A DIRECTED TEST

The trace is near-minimal, so the test is short, and it is exactly the stimulus a random generator was demonstrably not producing.

Travels with it: the property it violated, so the failure condition is the assertion rather than a hand-written expectation — and a cover on the scenario, so a later constraint change that quietly stops reaching it is visible.

AN UNREACHABILITY PROOF BECOMES A WAIVER'S JUSTIFICATION

Travels with it: the constraint set, as evidence, reviewed as seriously as the waiver it replaces — plus the flag on the coverage item that warns if the point is ever covered later.

AN ASSUMPTION BECOMES A CHECKER

The most valuable of the three, because it runs every night. **Travels with it:** the assumption register row it discharges, and the boundary cover.

Simulation → formal

A COVERAGE HOLE BECOMES A REACHABILITY QUESTION

A hole that has resisted three targeted tests is the classic candidate. Two of the three outcomes are decisive: a *witness* hands the stimulus engineer a recipe, and *unreachable* converts the hole into an exclusion with a proof behind it. **Inconclusive leaves the hole exactly where it was** — worth knowing before spending a week on it.

A LONG FAILING TRACE BECOMES A PROPERTY

Formulating the violated rule and running a bounded proof often returns a counterexample an order of magnitude shorter. **Travels with it:** the seed and run manifest of the original failure — the short trace is a *different execution*, and someone must confirm it is the same bug.

A REDUNDANT-STATE INVARIANT BECOMES A LEMMA

Exactly the facts a proof engine must be told to stop generating impossible starting states — and they cost nothing to hand over, because they are already written and already believed.

Staffing follows from the list: every handoff here needs someone who can read both kinds of report, so a project with exactly one such person has a bottleneck rather than a hybrid flow.

Evidence that names its own limits

The through-line. Chapter 13 buys a structural answer before a testbench exists. Chapter 14 changes the question from *did any run break this?* to *can anything break this?*, and pays for it in qualifiers. Chapter 15 packages the parts where the specification already exists. Chapter 16 spends both engines as one campaign.

WHAT PART IV DELIBERATELY LEFT OPEN

Chapter 17 takes metastability injection onto the emulator that can afford it. **Chapter 19** asks what survives at gate level, where X-pessimism gets worse before it gets better. And **Chapter 23** answers what Chapter 15's no-connection check cannot: not *does a path exist*, but *does the key influence anything observable along one that does*.

SEVEN SENTENCES WORTH LEAVING WITH

- A static gate is exhaustive over structure and silent about function: why it goes first, and why it is never the whole story.
- One bit needs metastability fixed; several bits need metastability *and* coherency fixed.
- A proof is a statement about a model, under assumptions, sometimes to a depth. Without those three it is a mood.
- A missing assumption arrives in red; an extra one announces nothing. Review the assume list, not the checkmarks.
- A bounded pass says the property cannot be violated *too soon*: carry the depth, and cover the antecedent under it.
- An app relocates the specification, it does not remove it — and one that mines the design proves by construction.
- Unify at the plan: rows closed out of rows planned has a denominator anyone at the review can name.

The chapters behind these slides

Chapter 13	Static Verification
Chapter 14	Formal Property Verification
Chapter 15	Formal Apps and Equivalence
Chapter 16	Hybrid Flows

The checkers, the assumption register and the sign-off tables sketched here appear there in full.